Modificações feitas no diagrama:
Padrão GRASP:

1. Controller (GameManager)
   Motivo: A classe GameManager pode agir como controller, fazendo com que o front-end só precise lidar com ela quando obter solicitação do usuário. Para isso, precisei colocar atributos que fazem com que GameManager tenha informações das peças, permitindo que ela gerencie os cliques e envie para as classes corretas.

   Adições:
   - comprarTileDoTurno(): Tile (O GameManager pede para o TileManager comprar a peça e guarda seu estado momentaneamente.
   - rotacionarTileAtual(): void
   - posicionarTileNoTabuleiro(x: int, y: int): boolean (Delega a validação do tile para TabuleiroManager e a inserção para Tabuleiro)
   - alocarMeepleNoTileAtual(idSubsegmento:int):boolean

   Exemplo:

```
public void rotacionarTileAtual() {
       //GameManager atua como o controller, delegando o trabalho de rotação de tile para o TileManager
    tileManager.rotacionarTileAtualDoBaralho();
}
```

Neste caso, o GameManager atua como Controller, o TileManager gerencia o estado da peça atual no turno e o Tile como especialista da informação, sendo responsável pela lógica de girar a si mesmo.

2. **Especialista da Informação**
   Motivo: Uma classe especialista contém as informações necessárias para resolver problemas a partir de métodos contidos em si mesma. As classes que podem agir como especialistas da informação são:
   a. Tile: Possui os atributos *subsegmentos: TipoTerreno[13]* e *meeplesAlocados[0..13]*. Sendo assim, ele é o especialista para girar seus subsegmentos e registrar/remover meeples de um subsegmento.
      i. Adição do método posicionarMeeple(meeple:Meeple, idSubsegmento: int): void (guarda o meeple na posição idSubsegmento do Tile)
      ii. Adição do método removerMeeple(idSubsegmento:int):Meeple (Devolve o meeple quando ocorre a pontuação)
   b. Meeple: Possui os atributos tipoMeeple: tipoMeeple[0..1], localizacaoNoTile:int, tileAlvo[0..1] e sabe quem é o Participante dono dele, portanto ele é o especialista em relação ao seu estado de alocação e sua classificação.
      i. Adição do método definirClassificacao(terreno: TipoTerreno): void (Como o Meeple conhece seu tipo, ele recebe tipoTerreno onde pisou e define a si mesmo sua própria classificação; se pisou em CIDADE vira CAVALEIRO, se CAMPO vira FAZENDEIRO, etc.)
      ii. Adição do método obterDono(): Participante (Necessário para adicionar pontuação ao Participante correto que possui o meeple em questão).

c. Tabuleiro: Possui o atributo tilesNoTabuleiro[0..72], sendo ele o responsável por obter os tiles adjacentes, já que ele possui a lista de tiles presentes.
    i. Adição do método obterTilesAdjacentes(x: int, y: int): List<Tile> (O especialista da informação busca seus vizinhos para facilitar a validação).
    ii. Adição do método estaVazio(x: int, y: int): boolean (Verifica rapidamente se o tile da posição (x, y) está vazio.)

3. **Criador (Participante: criadora de Meeple e TileManager: criadora de Tile)**
Motivo: Uma classe criadora é a responsável pela criação de uma nova instância de uma classe. Uma classe que agrega ou contém objetos de outra classe pode ser sua criadora. A classe Participante possui uma agregação com a classe Meeple, sendo assim, ela pode criar instâncias do tipo Meeple. Já a classe TileManager possui uma associação unidirecional com Tile, possuindo objetos Tile que formam o baralho do jogo (-baralho[0-72])
Adições:
    - Adição do método inicializarMeeples() na classe Participante (responsável por criar objetos do tipo Meeple).
    - Adição do método inicializarBaralho() na classe TileManager (responsável por criar objetos do tipo Tile)
    - Adição do método rotacionarTileAtualDoBaralho() na classe TileManager. Necessário para podermos fazer com que GameManeger se comunique com TileManager partir do método rotacionarTileAtual().
    - Adição do atributo -tileAtual do tipo Tile em TileManager para guardar a peça comprada no turno.